

Planar 3-Link Biped Kinetic and Potential Energies including D,C,G matrices according to Lagrange Formalism (MATLAB Code)

Ashish Kumar, Kiran Sairam Bethi Balagangadaran, Marlon Rivas, Prathyush Ravula

Coordinate Frame Convention

Inertia Coordinate Frame Convention

```
x (up)
^
|
|
0 ----> y (left)
```

Origin 0 is at the stance foot contact point.

+x axis points UPWARD.

+y axis points LEFT.

Angles are measured CLOCKWISE and taken as POSITIVE.

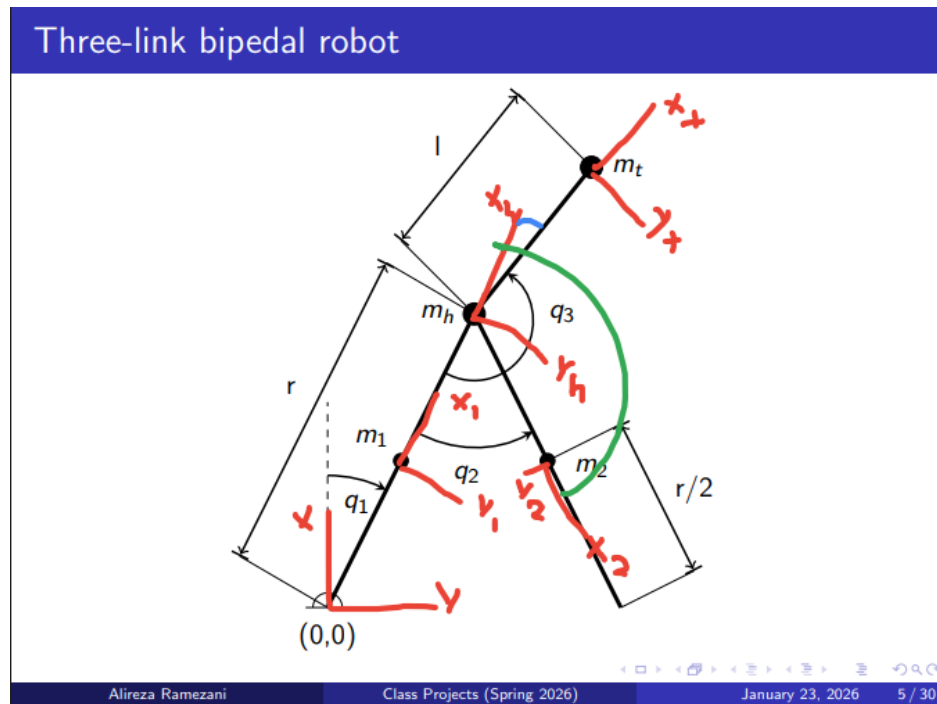


Figure 1: Coordinate frames

MATLAB Code Mini Project 1 and 2

```
clc; clear;
format short g

%% Symbols
syms q1 q2 q3 p_h p_v dp_h dp_v dq1 dq2 dq3 real
syms r m Mh Mt l g real

q = [q1; q2; q3];
dq = [dq1; dq2; dq3];

%% Helper transforms (2D homogeneous, 3x3)
Rot = @(th) [cos(th) -sin(th) 0;
             sin(th)  cos(th) 0;
             0        0      1];

TransX = @(a) [1 0 a;
               0 1 0;
               0 0 1];

%% --- Forward kinematics (SYMBOLIC) ---

% Stance leg: origin -> link1 start rotation, then translate along x by r
T_o_l1s = Rot(q1);
T_l1s_l1e = TransX(r);
T_o_l1e = T_o_l1s * T_l1s_l1e; % hip end (top of stance leg)

% Torso: from hip end rotate by (pi - q3) then translate by l
T_l1e_torso_s = Rot(pi - q3);
T_torso_s_torso_e = TransX(l);
T_o_torso_e = T_o_l1e * T_l1e_torso_s * T_torso_s_torso_e;

% m1: midpoint of stance leg
T_l1s_m1 = TransX(r/2);
T_o_m1 = T_o_l1s * T_l1s_m1;

% Swing leg: from hip end rotate by (pi - q2) then translate by r/2 to m2
T_l1e_sw_s = Rot(pi - q2);
T_sw_s_m2 = TransX(r/2);
T_sw_s_m2_e = TransX(r);
T_o_m2 = T_o_l1e * T_l1e_sw_s * T_sw_s_m2;
T_o_m2_e = T_o_l1e * T_l1e_sw_s * T_sw_s_m2_e;

%% --- Extract positions (2x1) ---
pHip = T_o_l1e(1:2, 3);
pTorso = T_o_torso_e(1:2, 3);
pM1 = T_o_m1(1:2, 3);
pM2 = T_o_m2(1:2, 3);
pM2End = T_o_m2_e(1:2, 3);
```

```

%% --- Jacobians (2x3) ---
J_Hip    = simplify(jacobian(pHip,    q));
J_Torso  = simplify(jacobian(pTorso,  q));
J_M1     = simplify(jacobian(pM1,     q));
J_M2     = simplify(jacobian(pM2,     q));
J_M2End   = simplify(jacobian(pM2End,  q));

% %% Display symbolic results
% disp("pHip = ");    disp(pHip);
% disp("pTorso = ");  disp(pTorso);
% disp("pM1 = ");     disp(pM1);
% disp("pM2 = ");     disp(pM2);
%
% disp("J_Hip = ");    disp(J_Hip);
% disp("J_Torso = ");  disp(J_Torso);
% disp("J_M1 = ");     disp(J_M1);
% disp("J_M2 = ");     disp(J_M2);

%% --- Optional: evaluate at your test values ---
r_val    = 1.0;
l_val    = 0.5;
q1_val   = pi/6;
q2_val   = 2*pi/3;
q3_val   = pi/4;

subs_list = [r, l, q1, q2, q3];
vals_list = [r_val, l_val, q1_val, q2_val, q3_val];

pHip_num  = double(subs(pHip,    subs_list, vals_list));
pTorso_num = double(subs(pTorso,  subs_list, vals_list));
pM1_num   = double(subs(pM1,     subs_list, vals_list));
pM2_num   = double(subs(pM2,     subs_list, vals_list));

J_Hip_num  = double(subs(J_Hip,    subs_list, vals_list));
J_Torso_num = double(subs(J_Torso,  subs_list, vals_list));
J_M1_num   = double(subs(J_M1,     subs_list, vals_list));
J_M2_num   = double(subs(J_M2,     subs_list, vals_list));

% disp("Numeric pHip, pTorso, pM1, pM2:");
% disp(pHip_num); disp(pTorso_num); disp(pM1_num); disp(pM2_num);
%
% disp("Numeric Jacobians:");
% disp(J_Hip_num); disp(J_Torso_num); disp(J_M1_num); disp(J_M2_num);

dq1_val = 0.5;
dq2_val = 0.3;
dq3_val = 0.2;

dpHip = J_Hip_num * [dq1_val; dq2_val; dq3_val];
dpTorso = J_Torso_num * [dq1_val; dq2_val; dq3_val];
dpM1 = J_M1_num * [dq1_val; dq2_val; dq3_val];
dpM2 = J_M2_num * [dq1_val; dq2_val; dq3_val];

% disp("Velocities in cartesian space:");

```

```

% disp(dpHip); disp(dpTorso); disp(dpM1); disp(dpM2);

% masses
m_val = 1.0;
Mh_val = 2.0;
Mt_val = 3.0;
M_tot = Mh_val + Mt_val + 2*m_val;
pCoM = (Mh_val*pHip + Mt_val*pTorso + m_val*pM1 + m_val*pM2)/M_tot;
J_PCoM = simplify(jacobian(pCoM, q));
% disp("J_PCoM = "); disp(J_PCoM);
% J_PCoM_num = double(subs(J_PCoM, subs_list, vals_list));
% pCoM_num = double(subs(pCoM, subs_list, vals_list));
% dpCoM = J_PCoM_num * [dq1_val; dq2_val; dq3_val];
% disp("CoM pose:");
% disp(pCoM_num)
% disp("CoM velocity:");
% disp(dpCoM)

% kinetic energy equations

D = ((m*J_M1'*J_M1)+(m*J_M2'*J_M2)+(Mt*J_Torso'*J_Torso)+(J_Hip'*Mh*J_Hip)
)

KE = 0.5*(dq')*(D)*dq

PE = g*((Mt*pTorso(1))+(Mh*pHip(1))+(m*pM1(1))+(m*pM2(1)))

coriolis_matrix(D, q, dq);

%TEST CODE BELOW FOR THE 3 FOR LOOPS

function C = coriolis_matrix(D, q, qdot)
% CORIOLIS_MATRIX Computes the Coriolis and centrifugal matrix C(q, qdot)
% using Christoffel symbols of the first kind.
%
% Inputs:
%   D      - n x n symbolic inertia/mass matrix (function of q)
%   q      - n x 1 symbolic vector of generalized coordinates
%   qdot   - n x 1 symbolic vector of generalized velocities
%
% Output:
%   C      - n x n symbolic Coriolis matrix

n = length(q); % number of generalized coordinates
C = sym(zeros(n, n));

C_logic = sym(zeros(n,n));
tic
for k = 1:n
    for j = 1:n
        c_kj = sym(0);
        for i = 1:n

```

```

        % Christoffel symbol: (1/2)*(dd_kj/dq_i + dd_ki/dq_j - dd_ij/
        dq_k)
        christoffel = (1/2) * ( diff(D(k,j), q(i)) ...
                                + diff(D(k,i), q(j)) ...
                                - diff(D(i,j), q(k)) );
        c_kj = c_kj + christoffel * qdot(i);
    end
    C(k,j) = simplify(c_kj);
end
end
elapsed_time = toc;
%disp("Brute force elapsed time");
%disp(elapsed_time);
disp("C_bruteforce")
disp(C)

global dict
dict = containers.Map('KeyType', 'char', 'ValueType', 'double');
tic
for k = 1:n
    for j = 1:n
        c_kj = sym(0);
        for i = 1:n
            christoffel = sym(0);
            if i ~= j
                key = [k j i];
                key_str = mat2str(key);
                if isKey(dict, key_str)
                    christoffel = dict(key);

                else
                    % Christoffel symbol: (1/2)*(dd_kj/dq_i + dd_ki/dq_j -
                    dd_ij/dq_k)
                    christoffel = (1/2) * ( diff(D(k,j), q(i)) ...
                                            + diff(D(k,i), q(j)) ...
                                            - diff(D(i,j), q(k)) );

                end
            else
                christoffel = (1/2) * ( diff(D(k,j), q(i)) ...
                                        + diff(D(k,i), q(j)) ...
                                        - diff(D(i,j), q(k)) );

            end

        end

        %Rest of Christoffel Symbols are not being print as they are
        0.
        if christoffel ~= 0

            christoffel

        end
    end
end

```

```

        c_kj = c_kj + christoffel * qdot(i);
    end
    C_logic(k,j) = simplify(c_kj);
end
end
elapsed_time = toc;
%disp("Logic elapsed time");
%disp(elapsed_time);
disp("C_logic: ")
disp(C_logic)

end

```

Results for Mini-project 2

```

D =

[Mt*(r*sin(q1) - l*sin(q1 - q3))^2 + Mt*(r*cos(q1) - l*cos(q1 - q3))^2 + (
    m*r^2*(cos(q1 - q2) - 2*cos(q1))^2/4 + Mh*r^2*cos(q1)^2 + (m*r^2*(sin(
    q1 - q2) - 2*sin(q1))^2/4 + Mh*r^2*sin(q1)^2 + (m*r^2*cos(q1)^2)/4 + (
    m*r^2*sin(q1)^2)/4, - (m*r^2*cos(q1 - q2)*(cos(q1 - q2) - 2*cos(q1)))/4
    - (m*r^2*sin(q1 - q2)*(sin(q1 - q2) - 2*sin(q1)))/4, Mt*l*sin(q1 - q3)
    *(r*sin(q1) - l*sin(q1 - q3)) + Mt*l*cos(q1 - q3)*(r*cos(q1) - l*cos(q1
    - q3))]

[
    - (m*r^2*cos(q1 - q2)*(cos(q1 - q2) - 2*cos(q1)))/4 - (m*r^2*sin(q1 -
    q2)*(sin(q1 - q2) - 2*sin(q1)))/4,
    (m*r^2*cos(q1 - q2)
    ^2)/4 + (m*r^2*sin(q1 - q2)^2)/4,
    0]

[
    Mt*l*sin(q1 - q3)*(r*sin(q1) - l*sin(q1 - q3)) + Mt*l*cos(q1 - q3)*(r*
    cos(q1) - l*cos(q1 - q3)),
    0,
    Mt*l^2*cos(q1 - q3
    )^2 + Mt*l^2*sin(q1 - q3)^2]

KE =

dq2*((dq2*((m*r^2*cos(q1 - q2)^2)/4 + (m*r^2*sin(q1 - q2)^2)/4))/2 - (dq1
    *((m*r^2*cos(q1 - q2)*(cos(q1 - q2) - 2*cos(q1)))/4 + (m*r^2*sin(q1 -
    q2)*(sin(q1 - q2) - 2*sin(q1)))/4))/2) + dq3*((dq1*(Mt*l*sin(q1 - q3)*(
    r*sin(q1) - l*sin(q1 - q3)) + Mt*l*cos(q1 - q3)*(r*cos(q1) - l*cos(q1 -
    q3))))/2 + (dq3*(Mt*l^2*cos(q1 - q3)^2 + Mt*l^2*sin(q1 - q3)^2))/2) +
    dq1*((dq3*(Mt*l*sin(q1 - q3)*(r*sin(q1) - l*sin(q1 - q3)) + Mt*l*cos(q1
    - q3)*(r*cos(q1) - l*cos(q1 - q3))))/2 + (dq1*(Mt*(r*sin(q1) - l*sin(
    q1 - q3))^2 + Mt*(r*cos(q1) - l*cos(q1 - q3))^2 + (m*r^2*(cos(q1 - q2)

```

```

- 2*cos(q1))^2)/4 + Mh*r^2*cos(q1)^2 + (m*r^2*(sin(q1 - q2) - 2*sin(q1)
)^2)/4 + Mh*r^2*sin(q1)^2 + (m*r^2*cos(q1)^2)/4 + (m*r^2*sin(q1)^2)/4))
/2 - (dq2*((m*r^2*cos(q1 - q2)*(cos(q1 - q2) - 2*cos(q1)))/4 + (m*r^2*
sin(q1 - q2)*(sin(q1 - q2) - 2*sin(q1)))/4))/2)

PE =

-g*(Mt*(l*(cos(q1)*cos(q3) + sin(q1)*sin(q3)) - r*cos(q1)) + m*((r*(cos(q1)
)*cos(q2) + sin(q1)*sin(q2)))/2 - r*cos(q1)) - (m*r*cos(q1))/2 - Mh*r*
cos(q1))

C_bruteforce
[(dq2*m*sin(q2)*r^2)/2 + Mt*dq3*l*sin(q3)*r, (m*r^2*sin(q2)*(dq1 - dq2))
/2, Mt*l*r*sin(q3)*(dq1 - dq3)]
[
    -(dq1*m*r^2*sin(q2))/2,
    0,
    0]
[
    -Mt*dq1*l*r*sin(q3),
    0,
    0]

christoffel =

(m*r^2*sin(q1 - q2)*(cos(q1 - q2) - 2*cos(q1)))/4 - (m*r^2*cos(q1 - q2)*(
sin(q1 - q2) - 2*sin(q1)))/4

christoffel =

Mt*l*cos(q1 - q3)*(r*sin(q1) - l*sin(q1 - q3)) - Mt*l*sin(q1 - q3)*(r*cos(
q1) - l*cos(q1 - q3))

christoffel =

(m*r^2*sin(q1 - q2)*(cos(q1 - q2) - 2*cos(q1)))/4 - (m*r^2*cos(q1 - q2)*(
sin(q1 - q2) - 2*sin(q1)))/4

christoffel =

(m*r^2*cos(q1 - q2)*(sin(q1 - q2) - 2*sin(q1)))/4 - (m*r^2*sin(q1 - q2)*(
cos(q1 - q2) - 2*cos(q1)))/4

christoffel =

Mt*l*cos(q1 - q3)*(r*sin(q1) - l*sin(q1 - q3)) - Mt*l*sin(q1 - q3)*(r*cos(
q1) - l*cos(q1 - q3))

christoffel =

```

```

Mt*l*sin(q1 - q3)*(r*cos(q1) - l*cos(q1 - q3)) - Mt*l*cos(q1 - q3)*(r*sin(
    q1) - l*sin(q1 - q3))

christoffel =

(m*r^2*cos(q1 - q2)*(sin(q1 - q2) - 2*sin(q1)))/4 - (m*r^2*sin(q1 - q2)*(
    cos(q1 - q2) - 2*cos(q1)))/4

christoffel =

Mt*l*sin(q1 - q3)*(r*cos(q1) - l*cos(q1 - q3)) - Mt*l*cos(q1 - q3)*(r*sin(
    q1) - l*sin(q1 - q3))

C_logic:
[(dq2*m*sin(q2)*r^2)/2 + Mt*dq3*l*sin(q3)*r, (m*r^2*sin(q2)*(dq1 - dq2))
    /2, Mt*l*r*sin(q3)*(dq1 - dq3)]
[
    -(dq1*m*r^2*sin(q2))/2,
    0,
    0]
[
    -Mt*dq1*l*r*sin(q3),
    0,
    0]

```